

گزارش کدنویسی‌های تمرین ۱ یادگیری ماشین

نام: امیر غرقابی

شماره دانشجویی: ۸۱۰۱۰۲۲۱۷

۶. گزارش مسئله طبقه بندی خوش خیم یا بدخیم بودن سرطان

آ) الگوریتم Naïve Bayes یک الگوریتم طبقه بندی بر پایه قانون بیز است. فرض این الگوریتم، مستقل صص بودن ویژگی‌های داده‌ها است که البته در دنیای واقعی فرض بسیاری سختی است زیرا ویژگی‌های بیشتر داده‌های دنیای واقعی به یکدیگر وابستگی دارند. ساختار این الگوریتم سه جز اصلی زیر را دارد:

- ویژگی‌های دیتا. بطور مثال در دیتاست سرطان سینه، ویژگی‌های مدنظر شامل شعاع و بافت توده سرطانی می‌شود.
- برچسب کلاس. در دیتاست سرطان سینه، برچسب‌ها عبارتند از خوش‌خیم و بدخیم
- مدل احتمال: که احتمال قرارگیری هر داده را برای هر کلاس محاسبه می‌کند و در نهایت برچسب کلاسی را که بیشترین احتمال را دارد به دیتا اختصاص می‌دهد.

دلیل استفاده از Naïve Bayes به جای طبقه بند Bayes، فرض استقلالی است که در نظر گرفته می‌شود که باعث سهولت در محاسبات شده و امکان پردازش دیتاهای بزرگ را با بازده بیشتری می‌دهد. مزیت Naïve bayes این است که با تعداد داده محدود هم می‌تواند عملکرد خوبی داشته باشد و بسیار سریع‌تر ترین شود.

استفاده از این طبقه بند هزینه‌هایی را هم در پی خواهد داشت. فرض استقلال ویژگی‌ها باعث می‌شود در مجموعه داده‌هایی که ویژگی‌ها به یکدیگر وابسته هستند دقت در مقایسه با مدل‌های پیچیده‌تر کاهش یابد.

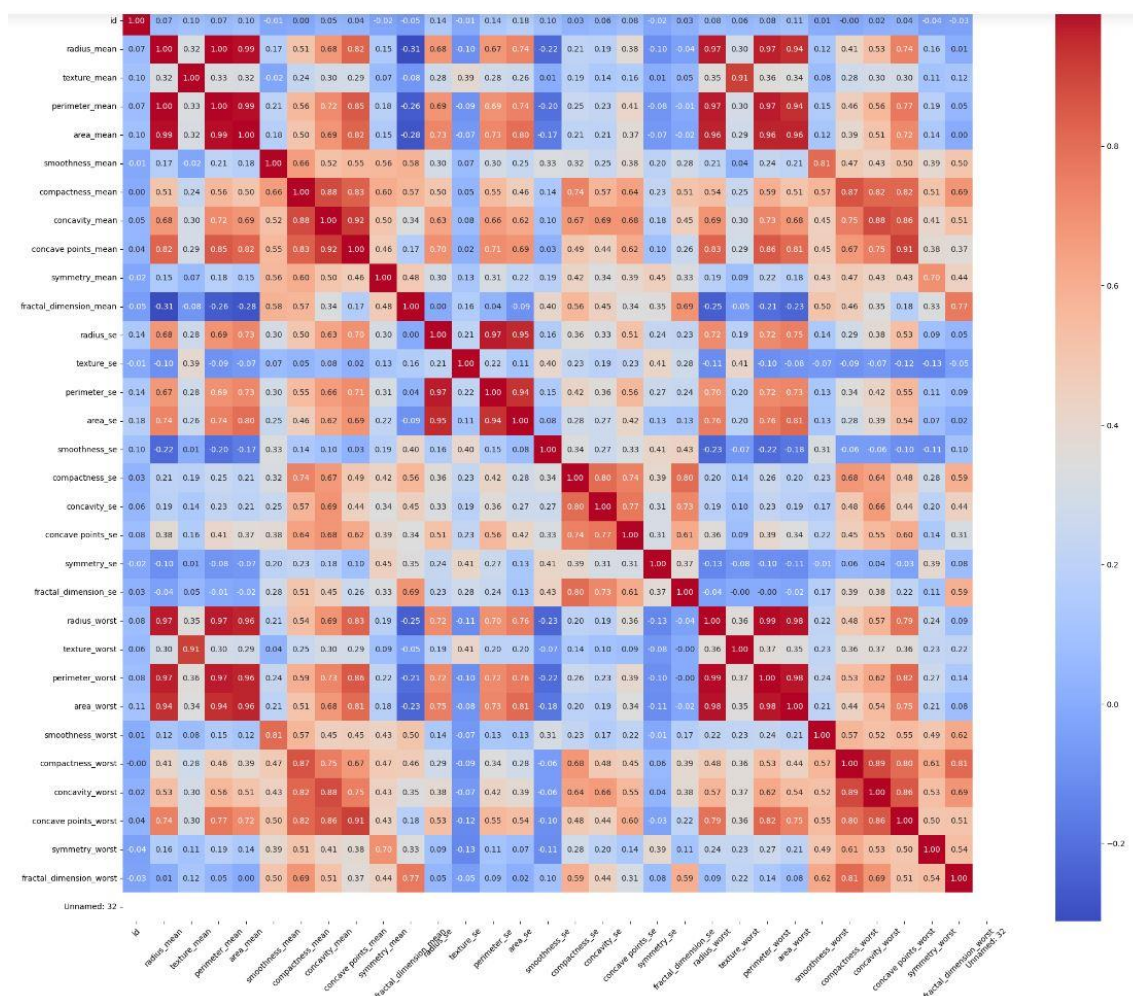
۱. طرح مسئله:

دیتاست شامل دیتای تصویری از توده سرطانی ۵۶۹ بیمار است که توده‌ی سرطانی ۳۵۷ نفر از آنها خوش‌خیم بوده و مابقی بدخیم. این مسئله با استفاده از الگوریتم Naïve bayes و در دو حالت ۱. بدون استفاده از کتابخانه Sk-learn و ۲. بدون استفاده از کتابخانه Sk-learn حل خواهد شد و نتایج در پایان مقایسه خواهند شد.

۱.۱. پاکسازی داده: در این دیتاست، بسیاری از ستون‌ها با یکدیگر همبستگی دارند

(correlation) بنابراین قبل از استفاده از داده، ستون‌هایی را که شبیه به یکدیگر هستند و

داده جدیدی اضافه نمیکنند را از داده اصلی حذف میکنیم. ماتریس heatmap در زیر قابل مشاهده است که در آن تنها ستون هایی با همبستگی بیش از ۹۰ درصد نمایش داده شده است:



بنابر ماتریس heatmap فوق، می توان ستون هایی را که با یکدیگر همبستگی بالایی (بالای ۹۰ درصد) دارند را حذف کرد.

این کار در تابع PreProcess_data انجام شده است. همچنین داده ها در همین تابع بصورت رندوم در کنار هم قرار گرفته اند.

2.1. طبقه بندی بدون استفاده از کتابخانه

پس از لود شدن فایل، عملیات پیش پردازش صورت می گیرد که شامل مراحل زیر است:

- حذف ستون ID و ستون های همبسته
- اختصاص عدد 1 برای Malignant و عدد 0 برای Benign
- شافل کردن دیتا. (تا بصورت رندوم در دو بخش ترین و تست قرار گیرند).

پس از پیش پردازش، داده ها به دو دسته ی ترین (۸۰٪) و تست (۲۰٪) قرار میگیرند. سپس با در نظر گرفتن برچسب هر داده به دو دسته 0 و 1 تقسیم میشوند. در ادامه میانگین و انحراف از معیار هر ستون

بطور جداگانه محاسبه می شود. پس از آن خلاصه از دیتای هر کلاس شامل میانگین و انحراف از معیار هر ستون را تولید می کند. با استفاده از تابع `Get_Prediction()`، برای دیتای تست الگوریتم Naïve Bays پیاده شده و برای هر داده ی تست برچسب یکی از دو کلاس 0 و 1 اختصاص داده خواهد شد. در واقع با استفاده از میانگین و انحراف از معیاری که در بخش قبل برای هر کلاس بدست آمده بود، احتمال قرار گرفتن هر داده در یکی از دو کلاس محاسبه می شود و ماکسیمم این احتمال، برچسب داده ی مورد نظر را مشخص می کند.

سپس با استفاده از برچسب های واقعی هر داده (که پس از حذف ID، در ستون اول قرار میگیرند) مقدار Accuracy و ماتریس confusion و همچنین Recall محاسبه می شود.

نتایج ترین کردن داده با درصد ۸۰٪ داده ترین و ۲۰٪ داده تست:

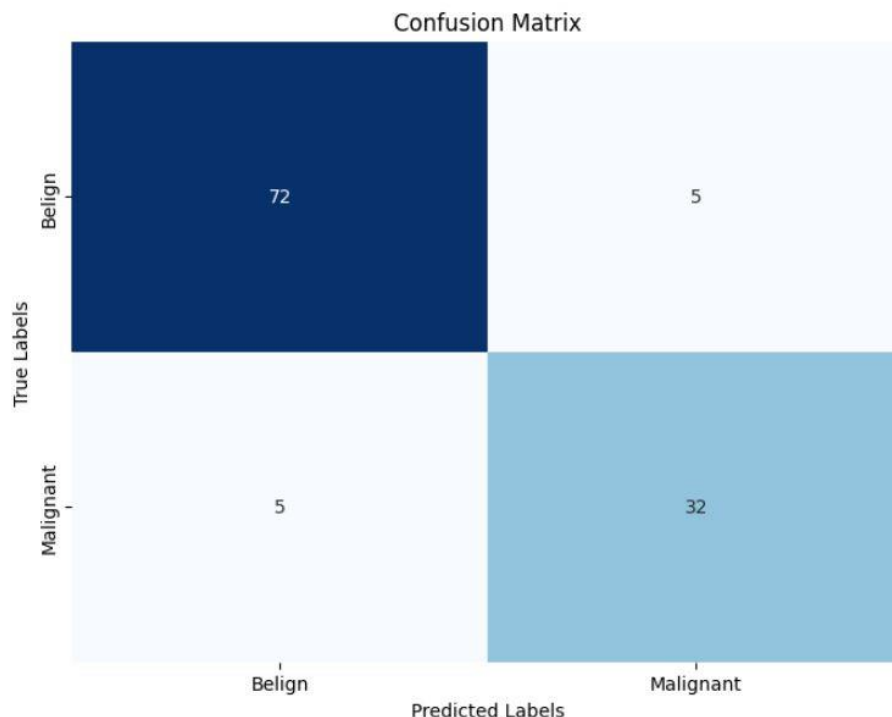
Self implemented Naive Bayes Results:

Confusion Matrix:

```
[[72  5]
 [ 5 32]]
```

Accuracy: 91.23%

Recall: 86.49%



2.2. طبقه بندی با استفاده از کتابخانه SK-Learn

اکنون با استفاده از کتابخانه Sk-Learn و کلاس GaussianNB نتیجه را محاسبه میکنیم.

لازم به ذکر است که داده طی مراحل یکسانی پیش پردازش شده و پس از شافل کردن آنها، داده های ترین و تست یکسانی برای هر دو حالت در نظر گرفته شده است تا مقایسه بهتری انجام شود.

Scikit-Learn Results:

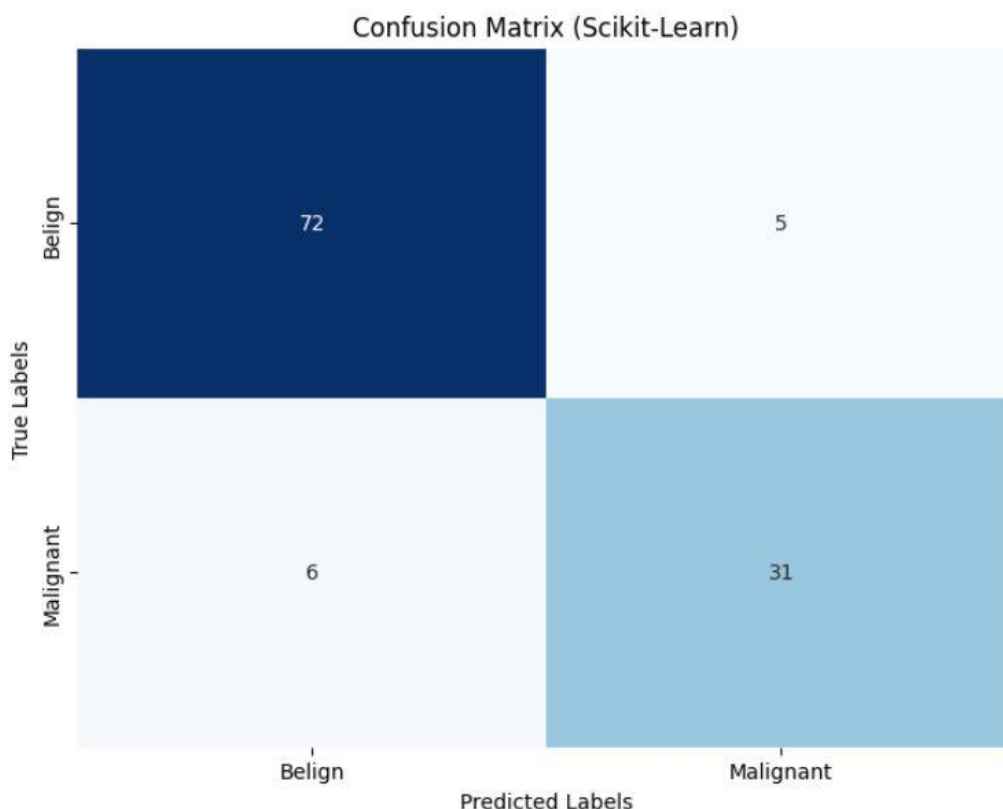
Confusion Matrix:

```
[[72  5]
```

```
 [ 6 31]]
```

Accuracy: 90.35%

Recall: 83.78%



نتایج فوق نشان تفاوت بسیار ناچیزی میان نتایج محاسبات دستی و کتابخانه sk-learn وجود دارد (در برخی از موارد شافل شدن دیتا، این تفاوت به 0 می رسد!) البته این تشابه زیاد منتظره است زیرا تعداد داده به نسبت زیاد است همچنین فرض گوسی بودن برای این داده نیز فرض معقولی است.

۷. گزارش مسئله طبقه بندی تصاویر جنگل و دریا

۱. طرح مسئله:

دیتاست دارای ۸۲ تصویر شامل ۴۲ تصویر از جنگل و ۴۰ تصویر از دریا است.

۲. روش طبقه بندی:

برای طبقه بندی این تصاویر از ویژگی بارز متمایزکننده ی این تصاویر یعنی رنگ آن ها استفاده خواهد شد. مراحل طبقه بندی در زیر قابل مشاهده است.

- تبدیل فرمت عکس ها به RGB
- استفاده از کتابخانه PIL برای گرفتن داده رنگی هر تصویر
- مقایسه پیکسل های سبز و آبی برای اختصاص برچسب به هر داده در تابع `classify_image` که آدرس تصویر را به عنوان ورودی دریافت می کند.

```
def classify_image(image_path):  
    image = Image.open(image_path)  
    image = image.convert("RGB")  
    red, green, blue = np.mean(image.getdata(band=0)), np.mean(image.getdata(band=1)), np.mean(image.getdata(band=2))  
    if green > blue and green > red:  
        return "Jungle"  
  
    else:  
        return "Sea"
```

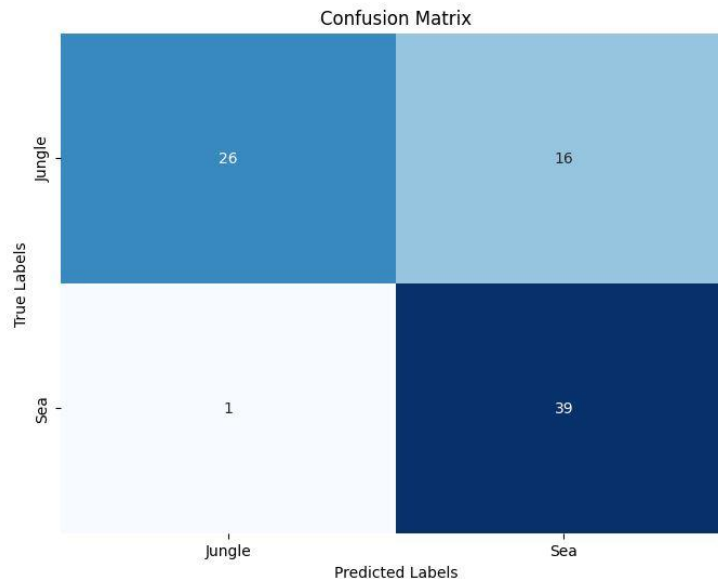
لازم به ذکر است که label هر تصویر از روی نام فایل آن تصویر قابل استخراج است. بطور مثال فایل تصاویر جنگل با حرف "J" شروع می شود در حالیکه فایل تصاویر دریا با حرف "S" شروع می شوند.

- استفاده از حلقه for برای بدست آوردن دیتای پیکسل هر عکس و اختصاص برچسبی متناظر با پیکسل های سبز و یا آبی.
- ترسیم ماتریس confusion و تولید مقادیر precision و recall

نتایج:

Results without considering Red color

```
Confusion Matrix:  
[[26 16]  
 [ 1 39]]  
Recall: 0.6190476190476191  
Precision: 0.9629629629629629
```



نتایج این ماتریس نشان می دهد که در تشخیص تصاویر جنگل پیشبینی های False positive زیادی وجود دارد. این اتفاق هم منتظره است زیرا تصاویری که با برچسب "جنگل" در دیتاست وجود دارند در رنگ های مختلفی (سبز و قرمز) هستند که باعث این خطا می شود. این مدل دارای precision=96% و Recall=61% است.

$$Precision = \frac{True\ positive}{True\ positive + False\ positive}$$

مقدار عددی precision نشانگر این است که این مدل در چند درصد از پیشبینی های مثبت خود صحیح پیشبینی کرده است که در این مورد عملکرد خوبی داشته است.

$$Recall = \frac{True\ positive}{True\ positive + False\ negative}$$

اما این مدل در Recall عملکرد خوبی نداشته است. که نشان می دهد negative false نسبت زیادی داشته است.

ماتریس فوق و همچنین مقدار Recall نشان می دهد که طبقه بند به خوبی عمل نمی کند. بنابراین برای طبقه بند شرط دیگری هم قرار می دهیم تا رنگ قرمز هم به عنوان تصویر جنگل در نظر گرفته شود. بدین ترتیب، تابع Classify_image() به شکل زیر در خواهد آمد.

```
def classify_image(image_path):  
    image = Image.open(image_path)  
    image = image.convert("RGB")  
    red, green, blue = np.mean(image.getdata(band=0)), np.mean(image.getdata(band=1)), np.mean(image.getdata(band=2))  
    if (green > blue and green > red) or red > blue:  
        return "Jungle"  
    else:  
        return "Sea"
```

نتایج جدید به شکل زیر خواهد بود:

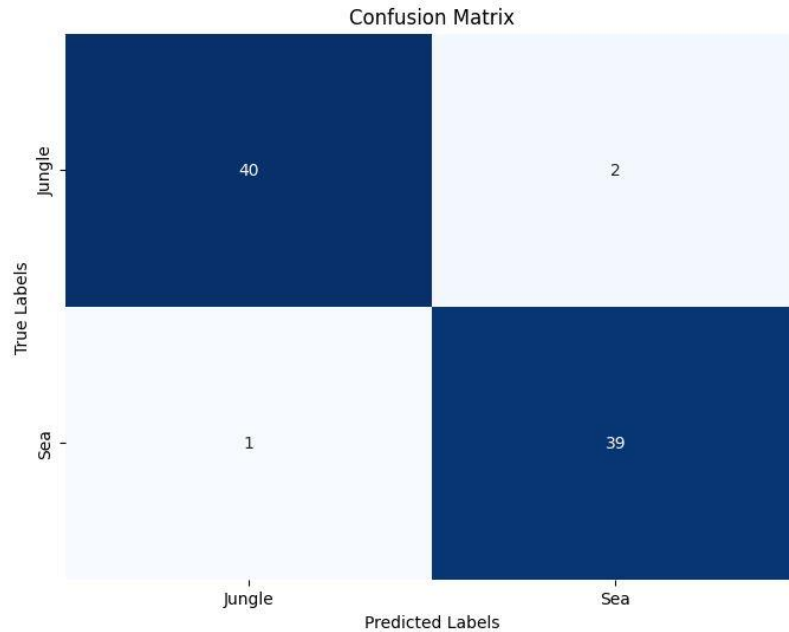
Results considering Red color

Confusion Matrix:

$\begin{bmatrix} 40 & 2 \\ 1 & 39 \end{bmatrix}$

Recall: 0.9523809523809523

Precision: 0.975609756097561



مشاهده می شود که این بار طبقه بند به خوبی عمل می کند و علاوه بر Recall، میزان صحت بالاتری را نیز دارد.

اگر چه در تصاویر زیر که نمایانگر جنگل هستند، به علت شباهت رنگی زیاد با دریا، به عنوان دریا تشخیص داده شده اند. که منتظره می باشد زیرا تنها از ویژگی رنگ برای طبقه بندی استفاده شده است.





همچنین تصویر زیر را که نمایانگر تصویری از دریا است نیز به عنوان جنگل تشخیص داده شده است:



این اتفاق هم مانند قسمت قبل منتظره است زیرا میزان رنگ سبز در این تصویر دریا زیاد بوده و طبقه بند را به اشتباه می اندازد.